Document number: P0604R0
Date: 2017-03-03
Project: Programming Language C++
Audience: Library Working Group, Library Evolution Working Group
Authors: Daniel Krügler, Pablo Halpern, Jonathan Wakely
Reply-to: Daniel Krügler

Resolving GB 55, US 84, US 85, US 86

Discussion

This proposal attempts to provide combined wording for the changes requested by the NB comments GB 55, US 84, US 85, and US 86 against C++17.

The general approach is to follow the suggested directions described by the issues:

1. **Required change:** Split *`INVOKE_R`* from being an additional *`INVOKE`* `"overload"`.
2. **Required change:** Rename and split `is_callable/is_nothrow_callable` into `is_invocable/is_nothrow_invocable/is_invocable_r/is_nothrow_invocable_r` (and associated types accordingly)
3. **Required change:** Change function type encoding of previous `is_callable/is_nothrow_callable/result_of` traits to conventional template type parameter lists.
4. **Optional change:** Deprecate `result_of` and introduce a new form without function type encoding whose name needs to be determined (Bikeshedding - Yeah!).

The changes in regard to `result_of` have been made optional, because this step increases the necessary draft changes significantly.

If we introduce a new replacement trait for `result_of`, this requires a new name. The following is a list of naming suggestions available at the point of writing this proposal:

1. `result_of_invoke` [Suggested by US 85]
2. `invoke_result` [Suggested by Jonathan Wakely]
3. `invocation_result` [Suggested by Jonathan Wakely]
4. `invocation` [Suggested by Jonathan Wakely]

> Without yet any concrete preferences expressed by the Committee, the proposal uses the name `invoke_result` as working title for that new trait.
>
> Nonetheless it should be noted that *all three* authors have independently expressed their preference for this name over the other choices.

Originally, Jonathan Wakely has made an alternative suggestion that could provide the return type of *`INVOKE()`*/`invoke()` without need of introducing a separate type trait for the result type. Instead the renamed `is_invocable` trait could in addition to its `type` member type provide another member type — possibly named `result_type` — conditionally, that is only defined if the trait evaluates to `true`.

The authors of this paper are *not* in favour of this alternative approach for the following reasons:

1. This would introduce a completely new way of retrieving information from traits, which is hard to explain and feels a bit like an inconsistent design.
2. The changes requested by the NB comments responded by this paper come very late, therefore the solution should not increase the chances of rejecting the whole paper just because of experimenal ideas that would better fit into an early investigation phase.
3. As Jonathan says: "One potential downside of that is that people might accidentally use `is_invocable::type` instead of `is_invocable::result_type`, which would always be present (it comes from the `bool_constant BaseCharacteristic` and is either `true_type` or `false_type`)."
4. For implementations the additional templates don't require extra work, since they can internally delegate to reusable entities.

5. Last but not least, it is easily possible to switch in the future to the alternative approach, in case that more good reasons are determined for such a design change.

Rationale

The authors of this paper recommend to apply all *four* suggested changes inspired by the NB comments as specified below for the following reasons:

1. **Rename `is_callable` to `is_invocable`:** `is_callable` would be the most natural name for a trait that answers the question whether a function call expression would be valid or not, which is a strict subset of the expressions, *`INVOKE`* supports, furthermore the changed name `is_invocable` much clearer expresses its meaning. Releasing the name `is_callable` allows us in the future to possibly introduce a pure `is_callable` trait (But this is not part of this proposal).

2. **Separating *`INVOKE_R`* from *`INVOKE`*:** *`INVOKE`* is an artificial construct and it is therefore very hard to logically discriminate a call of *`INVOKE`* *with* or *without* provided return type `R`. In every case where a pack expansion is provided, it is at least theoretically unclear whether the last parameter should be considered as part of the argument list of the first "overload" or as return type of the second "overload" (The fact that it is a type instead of a value helps, but the visual recognition is not easy). During the initial discussion of this topic, several people had expressed that they would prefer a *`INVOKE_R`* meta function that lists the return type *first* instead of as its *last* parameter (presumably because that has some similarities to a function template where this type would not be deduced). Daniel suggested a variant of the second form that would replace *`INVOKE_R`*`(R, f, args...)` by *`INVOKE`*`<R>(f, args...)` which is the form used below.

3. **Replace function type encoding of type lists in `is_callable` and `result_of`:** The function type encoding form used in these type traits is a relict from the pre-variadic template era of C++ where a single type is used to express a list of types. First, for both traits the natural interpretation of the encoded form `Fn(ArgTypes...)` looks as if `Fn` is used as return type, but it is only the callable object to which the arguments are provided. Second, contrary to the rest of all type traits which use a glvalue-based expression approach due to the *direct* usage of `declval` applied to the given type parameters, function type encoding is more natural for non-glvalues. In particular, parameter types are first *adjusted* via a special decay-like mechanism (8.3.5 [dcl.fct]), such that the actually provided types are often not the types on which the actual test is applied. Another problem is that function return types and argument types prevent some important type classes to be actually provided in the encoded form: Neither can argument types have an abstract class type nor *cv* `void` type, nor can function types or array types be used as "return type".

   As a workaround against the type transformation and restrictions mentioned above, user code needs to carefully ensure proper application of references to the arguments which causes problems for types that are non-*referenceable* types such as `void() const`, because instead af a silent SFINAE overload exclusion a compiler error will occur. Alisdair Meredith provided the following example of such an unpleasant situation on the Library reflector:

```
#include <type_traits>

struct Functor {
  template <typename T>
  void operator()(T&&) const {}
};

template <typename T>
void validate() {
  static_assert(!std::is_callable_v<Functor(T)>);
}

using abomination = int() const;

int main() {
  validate<abomination>();
}
```

   These subtleties of the function type encoding had lead to a series of Library issues in the past, such as the following ones:

   - LWG 2017: `std::reference_wrapper` makes incorrect usage of `std::result_of`
   - LWG 2021: Further incorrect usages of `result_of`
   - LWG 2767: `not_fn` *`call_wrapper`* can form invalid types

The Library Evolution Working Group had been asked to look at the design changes involved by this paper and the following lists the results of their evaluation:

Introduce *`invoke_result`*, by some name?

SF	F	N	A	SA
12	2	0	0	0

Deprecate `result_of`?

SF	F	N	A	SA

7	7	0	0	0

Name for *invoke_result*:

result_of_invoke_t<a, b, c>	0
invoke_result_t<a, b, c>	11
invocation_result_t<a, b, c>	7
invocation_t<a, b, c>	6
[invoke_result wins]	

Provide only `is_invocable<>::result_type` instead of `invoke_result`?

Unanimous no.

Provide both `invoke_result<>::type` and `is_invocable<>::result_type`?

SF	F	N	A	SA
0	2	6	5	1

The authors of this paper consider these results as a strong confirmation of the direction of their proposal.

Resolved Issues

If the proposed resolution would be accepted, the following library issues will be resolved:

Number	Description
2895	Passing function types to `result_of` and `is_callable`
2926	*INVOKE*`(f, t1, t2,... tN)` and *INVOKE*`(f, t1, t2,... tN, R)` are too similar
2927	Encoding a functor and argument types as a function signature for `is_callable` and `result_of` is fragile
2928	`is_callable` is not a good name

Proposed Resolution

At some places below, additional markup of the form

> [*Drafting notes*: whatever — *end drafting notes*]

is provided, which is *not* part of the normative wording, but is solely shown to provide additional information to the reader about the rationale of the concrete wording.

> **The below list of suggested changes is split into two halves: List (A) is an enumeration of suggested *minimalistic* wording changes and doesn't touch `result_of`, while list (B) separately lists the *additional* necessary changes when adjustments to `result_of` are considered.**

The proposed wording changes refer to the Standard C++ working draft N4640.

> [*Drafting notes*: Searching for current usages of *INVOKE*, the following locations are *unaffected* by the splitting of *INVOKE*<R>:
>
> 1. 20.5.3.5 [tuple.apply] p1
> 2. 20.7.7 [variant.visit] p2+p3
> 3. 20.14.3 [func.require] p1
> 4. 20.14.4 [func.invoke] p1
> 5. 20.14.5.4 [refwrap.invoke] p1
> 6. 20.14.10 [func.not_fn] p5+p6
> 7. 20.14.11.3 [func.bind.bind] p2+p3+p6
> 8. 20.14.12 [func.memfn] p1
> 9. Table 50 — "Other transformations" [Row for template `result_of`]

10. 30.3.2.2 [thread.thread.constr] p3+p5
11. 30.4.6.2 [thread.once.callonce] p1+p2
12. 30.6.9 [futures.async] p2+(3.1)+(3.2)

— *end drafting notes*]

A. *Minimalistic* suggestion (without any changes affecting `result_of`):

1. Modify 20.14.3 [func.require] as indicated:

   -2- Define *INVOKE*`<R>(f, t1, t2, ..., tN, R)` as `static_cast<void>(`*INVOKE*`(f, t1, t2, ..., tN))` if `R` is *cv* `void`, otherwise *INVOKE*`(f, t1, t2, ..., tN)` implicitly converted to `R`.

2. Modify 20.14.11.3 [func.bind.bind] as indicated:

   ```
   template<class R, class F, class... BoundArgs>
     unspecified bind(F&& f, BoundArgs&&... bound_args);
   ```

   [...]

   -7- *Returns:* A forwarding call wrapper `g` (20.14.3 [func.require]). The effect of $g(u_1, u_2, \ldots, u_M)$ shall be

   $$INVOKE\langle R\rangle(fd, \text{std::forward}\langle V_1\rangle(v_1), \text{std::forward}\langle V_2\rangle(v_2), \ldots, \text{std::forward}\langle V_N\rangle(v_N), R)$$

   where [...]

3. Modify 20.14.13.2 [func.wrap.func] as indicated:

   -2- A callable type (20.14.2 [func.def]) `F` is *Lvalue-Callable* for argument types `ArgTypes` and return type `R` if the expression *INVOKE*`<R>(declval<F&>(), declval<ArgTypes>()..., R)`, considered as an unevaluated operand (Clause 5 [expr]), is well formed (20.14.3 [func.require]).

4. Modify 20.14.13.2.4 [func.wrap.func.inv] as indicated:

   ```
   R operator()(ArgTypes... args) const;
   ```

   -1- *Returns:* *INVOKE*`<R>(f, std::forward<ArgTypes>(args)..., R)` (20.14.3 [func.require]), where `f` is the target object (20.14.2 [func.def]) of `*this`.

5. Modify 20.15.2 [meta.type.synop], header `<type_traits>` synopsis, as indicated:

   [*Drafting notes*: The newly introduced variable template constants are intentionally marked as `inline` to be consistent with the resolution bullet list (B2) from P0607R0 "Inline Variables for the Standard Library" which had been preferred by the Committee — *end drafting notes*]

   ```
   [...]
   // 20.15.6 [meta.rel], type relations
   [...]

   template <class, class R = void> struct is_callable; // not defined
   template <class Fn, class... ArgTypes, class R>
   struct is_callable<Fn(ArgTypes...), R>;
   template <class Fn, class... ArgTypes> struct is_invocable;
   template <class R, class Fn, class... ArgTypes> struct is_invocable_r;

   template <class, class R = void> struct is_nothrow_callable; // not defined
   template <class Fn, class... ArgTypes, class R>
   struct is_nothrow_callable<Fn(ArgTypes...), R>;
   template <class Fn, class... ArgTypes> struct is_nothrow_invocable;
   template <class R, class Fn, class... ArgTypes> struct is_nothrow_invocable_r;

   [...]

   // 20.15.6 [meta.rel], type relations
   [...]
   template <class T, class R = void> constexpr bool is_callable_v
     = is_callable<T, R>::value;
   template <class T, class R = void> constexpr bool is_nothrow_callable_v
     = is_nothrow_callable<T, R>::value;
   template <class Fn, class... ArgTypes> inline constexpr bool is_invocable_v
     = is_invocable<Fn, ArgTypes...>::value;
   template <class R, class Fn, class... ArgTypes> inline constexpr bool is_invocable_r_v
     = is_invocable_r<R, Fn, ArgTypes...>::value;
   template <class Fn, class... ArgTypes> inline constexpr bool is_nothrow_invocable_v
   ```

```
  = is_nothrow_invocable<Fn, ArgTypes...>::value;
template <class R, class Fn, class... ArgTypes> inline constexpr bool is_nothrow_invocable_r_v
  = is_nothrow_invocable_r<R, Fn, ArgTypes...>::value;
```
[…]

6. Modify 20.15.6 [meta.rel], Table 44 — "Type relationship predicates", as indicated:

Table 44 — Type relationship predicates

Template	Condition	Comments
[…]		
`template <class Fn, class... ArgTypes, class R> struct is_invocable~~callable<Fn(ArgTypes...), R>~~;`	The expression *INVOKE*`(declval<Fn>(), declval<ArgTypes>()..., R)` is well formed when treated as an unevaluated operand	`Fn, R,` and all types in the parameter pack `ArgTypes` shall be complete types, *cv* `void`, or arrays of unknown bound.
`template <class R, class Fn, class... ArgTypes> struct is_invocable_r;`	The expression *INVOKE*`<R>(declval<Fn>(), declval<ArgTypes>()...)` is well formed when treated as an unevaluated operand	`Fn, R,` and all types in the parameter pack `ArgTypes` shall be complete types, *cv* `void`, or arrays of unknown bound.
`template <class Fn, class... ArgTypes, class R> struct is_nothrow_invocable~~callable<Fn(ArgTypes...), R>~~;`	`is_invocable~~callable~~_v<Fn, ArgTypes...~~Fn(ArgTypes...), R~~>` is `true` and the expression *INVOKE*`(declval<Fn>(), declval<ArgTypes>()..., R)` is known not to throw any exceptions	`Fn, R,` and all types in the parameter pack `ArgTypes` shall be complete types, *cv* `void`, or arrays of unknown bound.
`template <class R, class Fn, class... ArgTypes> struct is_nothrow_invocable_r;`	`is_invocable_r_v<R, Fn, ArgTypes...>` is `true` and the expression *INVOKE*`<R>(declval<Fn>(), declval<ArgTypes>()...)` is known not to throw any exceptions	`Fn, R,` and all types in the parameter pack `ArgTypes` shall be complete types, *cv* `void`, or arrays of unknown bound.

7. Modify 30.6.10.1 [futures.task.members] as indicated:

```
template <class F>
packaged_task(F&& f);
template <class F, class Allocator>
packaged_task(allocator_arg_t, const Allocator& a, F&& f);
```

-2- *Requires:* *INVOKE*`<R>(f, t1, t2, ..., tN~~, R~~)`, where `t1, t2, ..., tN` are values of the corresponding types in `ArgTypes...`, shall be a valid expression. [...]

[...]

```
void operator()(ArgTypes... args);
```

-16- *Effects:* As if by *INVOKE*`<R>(f, t1, t2, ..., tN~~, R~~)`, where `f` is the stored task of `*this` and `t1, t2, ..., tN` are the values in `args...`. [...]

[...]

```
void make_ready_at_thread_exit(ArgTypes... args);
```

-19- *Effects:* As if by *INVOKE*`<R>(f, t1, t2, ..., tN~~, R~~)`, where `f` is the stored task and `t1, t2, ..., tN` are the values in `args...`. [...]

[...]

B. *Additional* suggestion (solely focussing of changes related to `result_of`):

1. Modify 20.14.1 [functional.syn], header `<functional>` synopsis, as indicated:

```
// 20.14.4 [func.invoke], invoke
template <class F, class... Args>
  invoke_result_t<F, Args...>~~result_of_t<F&&(Args&&...)>~~ invoke(F&& f, Args&&... args);
```

2. Modify 20.14.4 [func.invoke] as indicated:

```
template <class F, class... Args>
  invoke_result_t<F, Args...>~~result_of_t<F&&(Args&&...)>~~ invoke(F&& f, Args&&... args);
```

3. Modify 20.14.5 [refwrap], class template `reference_wrapper` synopsis, as indicated:

```
[…]
// invocation
template <class... ArgTypes>
invoke_result_t<T&, ArgTypes...>
~~result_of_t<T&(ArgTypes&&...)>~~
operator() (ArgTypes&&...) const;
[…]
```

4. Modify 20.14.5.4 [refwrap.invoke] as indicated:

```
template <class... ArgTypes>
  invoke_result_t<T&, ArgTypes...>
  ~~result_of_t<T&(ArgTypes&&...)>~~
    operator() (ArgTypes&&...) const;
```

5. Modify 20.14.10 [func.not_fn], *`call_wrapper`* synopsis, as indicated:

```
template<class... Args>
  auto operator()(Args&&...) &
    -> decltype(!declval<invoke_result_t<FD&, Args...>~~result_of_t<FD&(Args&&...)>~~>());

template<class... Args>
  auto operator()(Args&&...) const&
    -> decltype(!declval<invoke_result_t<const FD&, Args...>~~result_of_t<const FD&(Args&&...)>~~>());

template<class... Args>
  auto operator()(Args&&...) &&
    -> decltype(!declval<invoke_result_t<FD, Args...>~~result_of_t<FD(Args&&...)>~~>());

template<class... Args>
  auto operator()(Args&&...) const&&
    -> decltype(!declval<invoke_result_t<const FD, Args...>~~result_of_t<const FD(Args&&...)>~~>());
```

[...]

```
template<class... Args>
  auto operator()(Args&&...) &
    -> decltype(!declval<invoke_result_t<FD&, Args...>~~result_of_t<FD&(Args&&...)>~~>());
template<class... Args>
  auto operator()(Args&&...) const&
    -> decltype(!declval<invoke_result_t<const FD&, Args...>~~result_of_t<const FD&(Args&&...)>~~>());
```

-5- [...]

```
template<class... Args>
  auto operator()(Args&&...) &&
    -> decltype(!declval<invoke_result_t<FD, Args...>~~result_of_t<FD(Args&&...)>~~>());
template<class... Args>
  auto operator()(Args&&...) const&&
    -> decltype(!declval<invoke_result_t<const FD, Args...>~~result_of_t<const FD(Args&&...)>~~>());
```

-6- [...]

6. Modify 20.14.11.3 [func.bind.bind] as indicated:

```
template<class R, class F, class... BoundArgs>
  unspecified bind(F&& f, BoundArgs&&... bound_args);
```

[...]

-10- The values of the *bound arguments* [...] :

(10.1) — [...]

(10.2) — if the value of `is_bind_expression_v<TD`$_i$`>` is `true`, the argument is `td`$_i$`(std::forward<U`$_j$`>(u`$_j$`)...)` and its type `V`$_i$ is `invoke_result_t<TD`$_i$ *cv* `&, U`$_j$`...>&&` ~~`result_of_t<TD`$_i$ *cv* `& (U`$_j$`&&...)>&&`~~;

(10.3) — [...]

7. Modify 20.15.2 [meta.type.synop], header `<type_traits>` synopsis, as indicated:

> [*Drafting notes*: The changes below additionally fix an existing inconsistent usage of class-key `class` instead of `struct`. Albeit both are equivalent as of the letters of the core language, there are compilers that mangle both forms differently. — *end drafting notes*]

```
// 20.15.7.6 [meta.trans.other], other transformations
[…]
~~template <class> class result_of; // not defined~~
~~template <class F, class... ArgTypes> class result_of<F(ArgTypes...)>;~~
template <class Fn, class... ArgTypes> struct invoke_result;
```

```
[…]

// 20.15.7.6 [meta.trans.other], other transformations
[…]
template <class T>
  using result_of_t = typename result_of<T>::type;
template <class Fn, class... ArgTypes>
  using invoke_result_t = typename invoke_result<Fn, ArgTypes...>::type;
[…]
```

8. Modify 20.15.7.6 [meta.trans.other], Table 50 — "Other transformations" and the following example, as indicated:

Table 50 — Other transformations

Template	Comments
[…]	
`template <class Fn, class... ArgTypes> struct invoke_result~~result_of<Fn(ArgTypes...)>~~;`	If the expression *INVOKE*`(declval<Fn>(), declval<ArgTypes>()...)` is well formed when treated as an unevaluated operand (Clause 5 [expr]), the member typedef type shall name the type `decltype(`*INVOKE*`(declval<Fn>(), declval<ArgTypes>()...))`; otherwise, there shall be no member type. Access checking is performed as if in a context unrelated to `Fn` and `ArgTypes`. […] *Requires:* `Fn` and all types in the parameter pack `ArgTypes` shall be complete types, *cv* `void`, or arrays of unknown bound.

[…]

-5- [*Example:* Given these definitions:

```
using PF1 = bool (&)();
using PF2 = short (*)(long);

struct S {
  operator PF2() const;
  double operator()(char, int&);
  void fn(long) const;
  char data;
};

using PMF = void (S::*)(long) const;
using PMD = char S::*;
```

the following assertions will hold:

```
static_assert(is_same_v<invoke_result_t<S, int>~~result_of_t<S(int)>~~, short>, "Error!");
static_assert(is_same_v<invoke_result_t<S&, unsigned char, int&>~~result_of_t<S&(unsigned char, int&)>~~, double>, "Error!");
static_assert(is_same_v<invoke_result_t<PF1>~~result_of_t<PF1()>~~, bool>, "Error!");
static_assert(is_same_v<invoke_result_t<PMF, unique_ptr<S>, int>~~result_of_t<PMF(unique_ptr<S>, int)>~~, void>, "Error!");
static_assert(is_same_v<invoke_result_t<PMD, S>~~result_of_t<PMD(S)>~~, char&&>, "Error!");
static_assert(is_same_v<invoke_result_t<PMD, const S*>~~result_of_t<PMD(const S*)>~~, const char&>, "Error!");
```

— *end example*]

9. Modify 30.6.2 [future.syn], header `<future>` synopsis, as indicated:

```
[…]
template <class F, class... Args>
  future<invoke_result_t<decay_t<F>, decay_t<Args>...>~~result_of_t<decay_t<F>(decay_t<Args>...)>~~>
  async(F&& f, Args&&... args);
template <class F, class... Args>
  future<invoke_result_t<decay_t<F>, decay_t<Args>...>~~result_of_t<decay_t<F>(decay_t<Args>...)>~~>
  async(launch policy, F&& f, Args&&... args);
```

10. Modify 30.6.9 [futures.async] as indicated:

```
template <class F, class... Args>
  future<invoke_result_t<decay_t<F>, decay_t<Args>...>~~result_of_t<decay_t<F>(decay_t<Args>...)>~~>
  async(F&& f, Args&&... args);
template <class F, class... Args>
  future<invoke_result_t<decay_t<F>, decay_t<Args>...>~~result_of_t<decay_t<F>(decay_t<Args>...)>~~>
  async(launch policy, F&& f, Args&&... args);
```

-2- […]

[…]

-4- *Returns:* An object of type `future<`invoke_result_t<decay_t<F>, decay_t<Args>...>~~result_of_t<decay_t<F>~~

~~(decay_t<Args>...)~~> that refers to the shared state created by this call to `async`. [...]

11. Modify D.11 [depr.meta.types], "Deprecated type traits" as indicated:

> [*Drafting notes*: The changes below additionally fix an existing inconsistent usage of class-key `class` instead of `struct`, see above.]

-1- The header `<type_traits>` has the following addition:

```
namespace std {
  template <class T> struct is_literal_type;
  template <class T> constexpr bool is_literal_type_v = is_literal_type<T>::value;
  template <class> struct result_of; // not defined
  template <class Fn, class... ArgTypes> struct result_of<Fn(ArgTypes...)>;
  template <class T> using result_of_t = typename result_of<T>::type;
}
```

-2- *Requires:* For `is_literal_type`, `remove_all_extents_t<T>` shall be a complete type or *cv* `void`. For `result_of<Fn(ArgTypes...)>`, `Fn` and all types in the parameter pack `ArgTypes` shall be complete types, *cv* `void`, or arrays of unknown bound.

-3- *Effects:* `is_literal_type` has a base-characteristic of `true_type` if `T` is a literal type (3.9), and a base-characteristic of `false_type` otherwise. The partial specialization `result_of<Fn(ArgTypes...)>` is a `TransformationTrait` whose member typedef `type` shall be defined if and only `invoke_result<Fn, ArgTypes...>::type` is defined. If `type` is defined, it shall name the same type as `invoke_result_t<Fn, ArgTypes...>`.

Feature-testing Macro

For the purposes of SG10, this paper recommends to change the originally suggested macro name `__cpp_lib_is_callable` to `__cpp_lib_is_invocable`.